

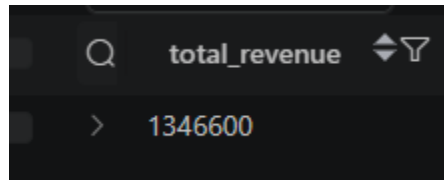
E-Commerce SQL Analysis

SQL Queries

1. Total Revenue

Calculates the sum of all order values to determine overall revenue.

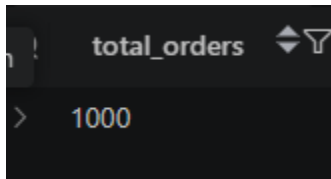
```
SELECT SUM(total_price) AS total_revenue  
FROM sales_data;
```



2. Total Orders

Counts the total number of orders placed.

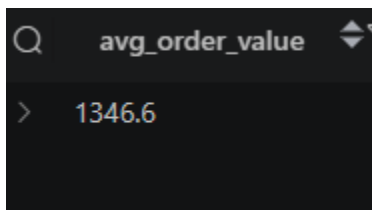
```
SELECT COUNT(*) AS total_orders  
FROM sales_data;
```



3. Average Order Value

Computes the mean value per order, rounded to two decimal places.

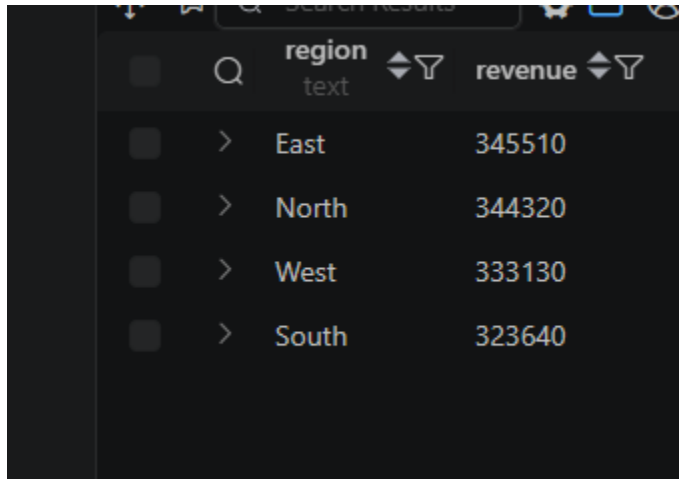
```
SELECT ROUND(AVG(total_price), 2) AS avg_order_value  
FROM sales_data;
```



4. Revenue by Region

Breaks down total revenue by geographic region, sorted highest first.

```
SELECT
    region,
    SUM(total_price) AS revenue
FROM sales_data
GROUP BY region
ORDER BY revenue DESC;
```



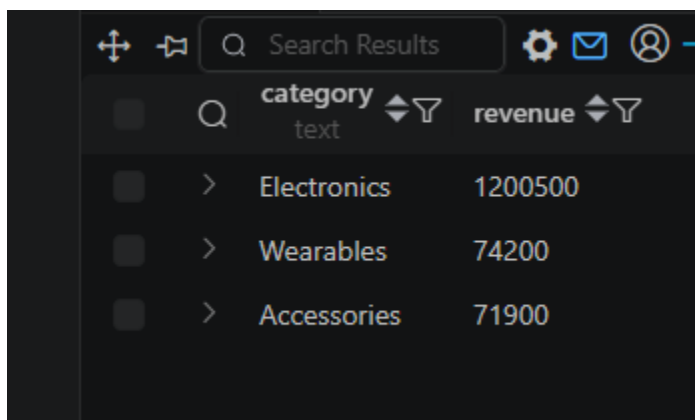
A screenshot of a database query result interface. At the top, there is a search bar labeled 'Search Results' with a magnifying glass icon. Below it, the table has two columns: 'region' (with a text filter icon) and 'revenue' (with a sort icon). The data is sorted in descending order of revenue. The regions listed are East, North, West, and South.

region	revenue
East	345510
North	344320
West	333130
South	323640

5. Revenue by Category

Shows total revenue for each product category.

```
SELECT
    category,
    SUM(total_price) AS revenue
FROM sales_data
GROUP BY category
ORDER BY revenue DESC;
```



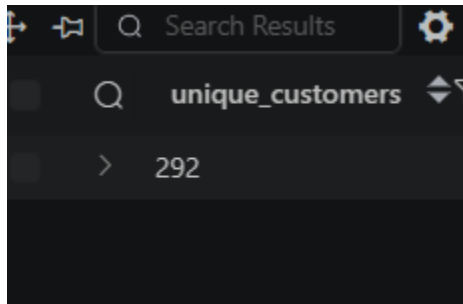
A screenshot of a database query result interface. At the top, there is a search bar labeled 'Search Results' with a magnifying glass icon. Below it, the table has two columns: 'category' (with a text filter icon) and 'revenue' (with a sort icon). The data is sorted in descending order of revenue. The categories listed are Electronics, Wearables, and Accessories.

category	revenue
Electronics	1200500
Wearables	74200
Accessories	71900

6. Count Unique Customers

Returns the number of distinct customers who have placed at least one order.

```
SELECT COUNT(DISTINCT customer_id) AS unique_customers
FROM sales_data;
```



7. Top 10 Customers by Spending

Lists the ten highest-spending customers with their total expenditure.

```
SELECT
    customer_id,
    SUM(total_price) AS total_spent
FROM sales_data
GROUP BY customer_id
ORDER BY total_spent DESC
LIMIT 10;
```

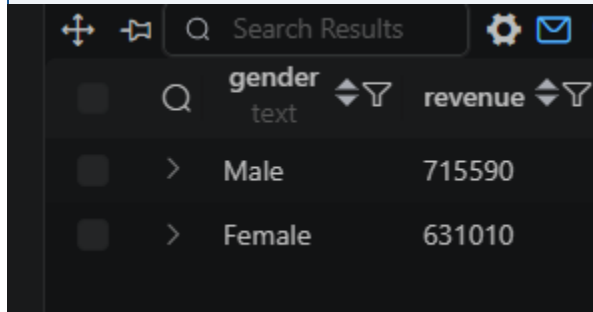
A screenshot of a SQL query result in a dark-themed interface. The result is a table with two columns: 'customer_id' and 'total_spent'. The table is sorted by 'total_spent' in descending order. The first row is 'CUST0204' with a total spent of '29100'. The last row is 'CUST0293' with a total spent of '14200'. Each row has a checkbox and a right arrow icon to its left.

	customer_id	total_spent
	text	
☐	> CUST0204	29100
☐	> CUST0095	22440
☐	> CUST0131	21180
☐	> CUST0139	18350
☐	> CUST0281	17850
☐	> CUST0144	17320
☐	> CUST0200	16700
☐	> CUST0197	16150
☐	> CUST0064	14600
☐	> CUST0293	14200

8. Revenue by Gender

Compares total revenue segmented by customer gender.

```
SELECT
  gender,
  SUM(total_price) AS revenue
FROM sales_data
GROUP BY gender;
```



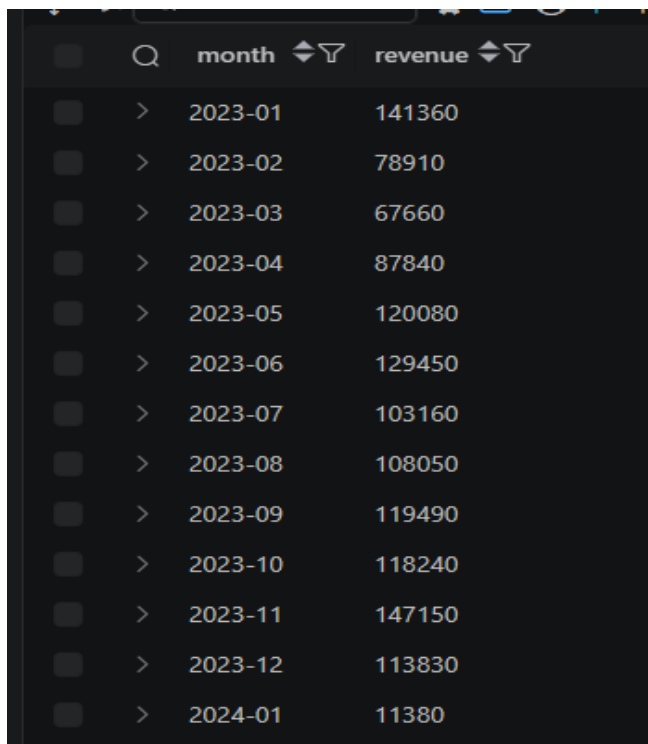
A screenshot of a database query result interface. At the top, there is a search bar with the text 'Search Results' and a magnifying glass icon. Below the search bar, the query results are displayed in a table format. The table has two columns: 'gender' and 'revenue'. The 'gender' column is labeled 'text' and has a dropdown arrow. The 'revenue' column has a dropdown arrow. The table contains two rows: 'Male' with a revenue of 715590, and 'Female' with a revenue of 631010. Each row has a checkbox to its left and a right-pointing arrow.

gender	revenue
Male	715590
Female	631010

9. Monthly Revenue Trend

Aggregates revenue month by month to visualize seasonal trends.

```
SELECT
  DATE_FORMAT(order_date, '%Y-%m') AS month,
  SUM(total_price) AS revenue
FROM sales_data
GROUP BY DATE_FORMAT(order_date, '%Y-%m')
ORDER BY month;
```



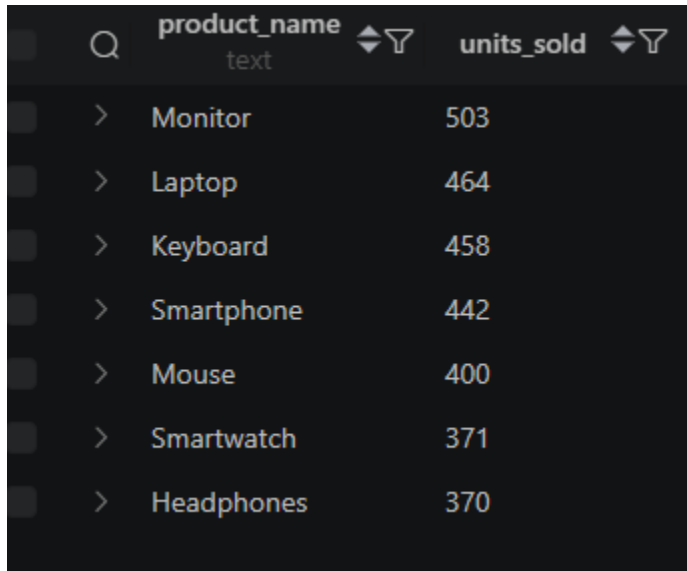
A screenshot of a database query result interface showing monthly revenue trends. The table has two columns: 'month' and 'revenue'. The 'month' column is labeled with the format '%Y-%m' and has a dropdown arrow. The 'revenue' column has a dropdown arrow. The table contains 13 rows, representing the months from January 2023 to January 2024. Each row has a checkbox to its left and a right-pointing arrow.

month	revenue
2023-01	141360
2023-02	78910
2023-03	67660
2023-04	87840
2023-05	120080
2023-06	129450
2023-07	103160
2023-08	108050
2023-09	119490
2023-10	118240
2023-11	147150
2023-12	113830
2024-01	11380

10. Most Popular Products

Ranks products by total units sold.

```
SELECT
    product_name,
    SUM(quantity) AS units_sold
FROM sales_data
GROUP BY product_name
ORDER BY units_sold DESC;
```

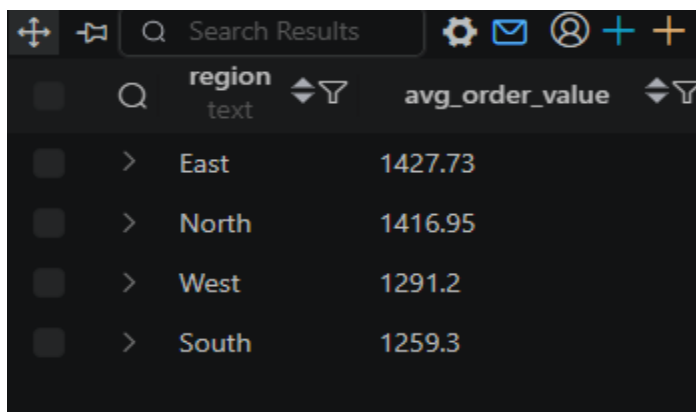


	product_name text	units_sold
>	Monitor	503
>	Laptop	464
>	Keyboard	458
>	Smartphone	442
>	Mouse	400
>	Smartwatch	371
>	Headphones	370

11. Average Order Value by Region

Compares mean order value across different regions.

```
SELECT
    region,
    ROUND(AVG(total_price), 2) AS avg_order_value
FROM sales_data
GROUP BY region
ORDER BY avg_order_value DESC;
```

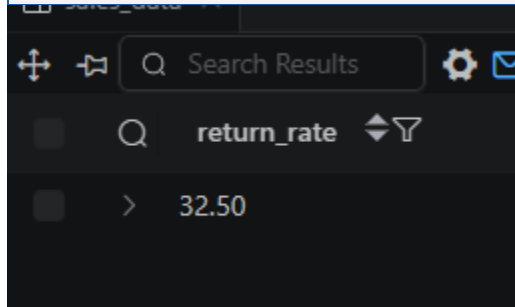


	region text	avg_order_value
>	East	1427.73
>	North	1416.95
>	West	1291.2
>	South	1259.3

12. Return Rate

Calculates the percentage of orders that were returned.

```
SELECT
  ROUND(
    100.0 * SUM(CASE WHEN shipping_status = 'Returned' THEN 1 ELSE 0
  END)
    / COUNT(*),
    2
  ) AS return_rate
FROM sales_data;
```



13. Products with Highest Return Rate

Identifies which products have the most returns, helping target quality improvements.

```
SELECT
  product_name,
  COUNT(*) AS total_orders,
  SUM(CASE WHEN shipping_status = 'Returned' THEN 1 ELSE 0 END) AS
  returned_orders,
  ROUND(
    100.0 * SUM(CASE WHEN shipping_status = 'Returned' THEN 1 ELSE 0
  END)
    / COUNT(*),
    2
  ) AS return_rate
FROM sales_data
GROUP BY product_name
ORDER BY return_rate DESC;
```

product_name text	total_orders	returned_orders	return_rate
Laptop	154	54	35.06
Headphones	120	41	34.17
Mouse	136	46	33.82
Smartwatch	122	41	33.61
Keyboard	145	45	31.03
Smartphone	142	44	30.99
Monitor	181	54	29.83

14. Rank Customers by Revenue

Uses a window function to assign rank positions to customers based on total spending.

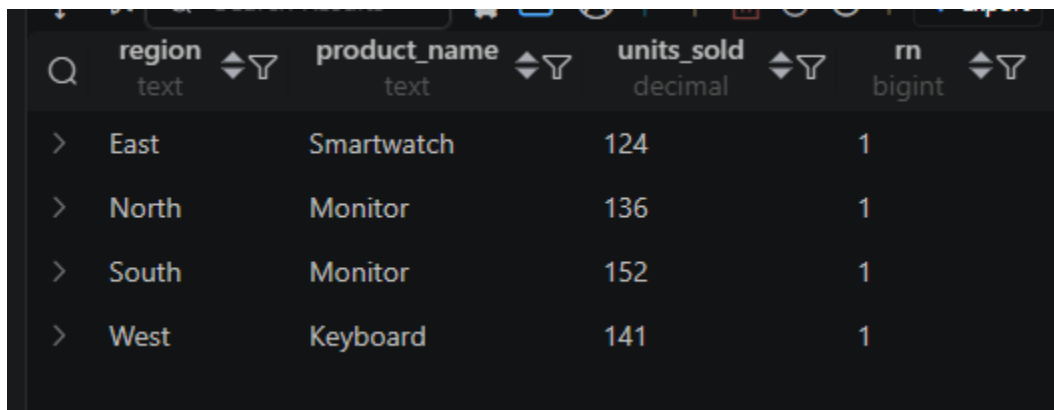
```
SELECT
  customer_id,
  SUM(total_price) AS revenue,
  RANK() OVER (
    ORDER BY SUM(total_price) DESC
  ) AS customer_rank
FROM sales_data
GROUP BY customer_id LIMIT 20;
```

	Q	customer_id text	revenue	customer_rank
☐	>	CUST0204	29100	1
☐	>	CUST0095	22440	2
☐	>	CUST0131	21180	3
☐	>	CUST0139	18350	4
☐	>	CUST0281	17850	5
☐	>	CUST0144	17320	6
☐	>	CUST0200	16700	7
☐	>	CUST0197	16150	8
☐	>	CUST0064	14600	9
☐	>	CUST0293	14200	10
☐	>	CUST0160	13750	11
☐	>	CUST0129	13460	12
☐	>	CUST0031	12800	13
☐	>	CUST0203	12480	14
☐	>	CUST0176	12400	15
☐	>	CUST0138	12350	16
☐	>	CUST0156	12000	17
☐	>	CUST0106	11150	18
☐	>	CUST0178	11100	19
☐	>	CUST0133	11100	19

15. Top-Selling Product in Each Region

Uses a CTE with ROW_NUMBER to find the best-selling product per region.

```
WITH product_sales AS (  
    SELECT  
        region,  
        product_name,  
        SUM(quantity) AS units_sold,  
        ROW_NUMBER() OVER (  
            PARTITION BY region  
            ORDER BY SUM(quantity) DESC  
        ) AS rn  
    FROM sales_data  
    GROUP BY region, product_name  
)  
SELECT *  
FROM product_sales  
WHERE rn = 1;
```



The screenshot shows a database query result with four columns: region, product_name, units_sold, and rn. The data is as follows:

region	product_name	units_sold	rn
East	Smartwatch	124	1
North	Monitor	136	1
South	Monitor	152	1
West	Keyboard	141	1

16. Customers Spending Above Average

Filters for customers whose total spend exceeds the overall customer average using a subquery.

```
WITH customer_spending AS (  
    SELECT  
        customer_id,  
        SUM(total_price) AS total_spent  
    FROM sales_data  
    GROUP BY customer_id  
)  
SELECT *  
FROM customer_spending  
WHERE total_spent > (  
    SELECT AVG(total_spent)  
    FROM customer_spending  
) ;
```

	customer_id text	total_spent double
>	CUST0046	7760
>	CUST0173	9250
>	CUST0159	6650
>	CUST0023	10450
>	CUST0201	8460
>	CUST0283	5840
>	CUST0199	6340
>	CUST0276	9230
>	CUST0139	18350
>	CUST0293	14200
>	CUST0196	6590
>	CUST0242	4800
>	CUST0264	6150
>	CUST0053	7550
>	CUST0288	9750
>	CUST0124	6000
>	CUST0298	5420
>	CUST0211	5640
>	CUST0060	6900
>	CUST0282	10600
>	CUST0299	10060
>	CUST0064	14600
>	CUST0247	5600
>	CUST0178	11100

17. Revenue Contribution by Region

Shows each region's revenue along with its percentage share of total revenue.

```
SELECT
    region,
    SUM(total_price) AS revenue,
    ROUND(
        100.0 * SUM(total_price) /
        SUM(SUM(total_price)) OVER (),
        2
    ) AS contribution_pct
FROM sales_data
GROUP BY region
ORDER BY revenue DESC;
```

	Q	region text	revenue	contribution_pct
	>	East	345510	25.66
	>	North	344320	25.57
	>	West	333130	24.74
	>	South	323640	24.03

18. Month-over-Month Growth

Calculates the percentage change in revenue compared to the previous month using LAG().

```
WITH monthly_sales AS (
    SELECT
        DATE_FORMAT(order_date, '%Y-%m') AS month,
        SUM(total_price) AS revenue
    FROM sales_data
    GROUP BY DATE_FORMAT(order_date, '%Y-%m')
)
SELECT
    month,
    revenue,
    LAG(revenue) OVER (ORDER BY month) AS prev_month,
    ROUND(
        ((revenue - LAG(revenue) OVER (ORDER BY month))
        / LAG(revenue) OVER (ORDER BY month)) * 100,
        2
    ) AS growth_pct
FROM monthly_sales
ORDER BY month;
```

	month varchar	revenue double	prev_month double	growth_pct double
>	2023-01	141360	(NULL)	(NULL)
>	2023-02	78910	141360	-44.18
>	2023-03	67660	78910	-14.26
>	2023-04	87840	67660	29.83
>	2023-05	120080	87840	36.7
>	2023-06	129450	120080	7.8
>	2023-07	103160	129450	-20.31
>	2023-08	108050	103160	4.74
>	2023-09	119490	108050	10.59
>	2023-10	118240	119490	-1.05
>	2023-11	147150	118240	24.45
>	2023-12	113830	147150	-22.64
>	2024-01	11380	113830	-90

19. Pareto Analysis (Top 20% Customers)

Retrieves the top 58 customers by revenue to support 80/20 Pareto analysis.

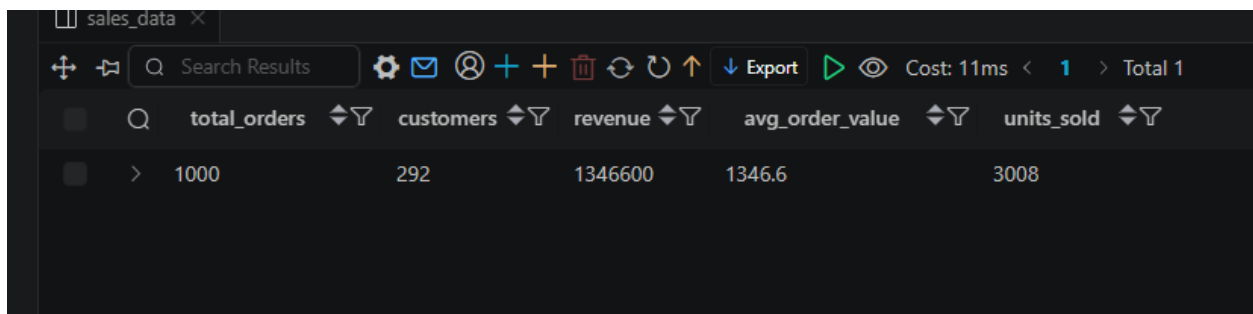
```
WITH customer_sales AS (  
  SELECT  
    customer_id,  
    SUM(total_price) AS revenue  
  FROM sales_data  
  GROUP BY customer_id  
)  
SELECT *  
FROM customer_sales  
ORDER BY revenue DESC  
LIMIT 58;
```

	customer_id text	revenue double
>	CUST0204	29100
>	CUST0095	22440
>	CUST0131	21180
>	CUST0139	18350
>	CUST0281	17850
>	CUST0144	17320
>	CUST0200	16700
>	CUST0197	16150
>	CUST0064	14600
>	CUST0293	14200
>	CUST0160	13750
>	CUST0129	13460
>	CUST0031	12800
>	CUST0203	12480
>	CUST0176	12400
>	CUST0138	12350
>	CUST0156	12000
>	CUST0106	11150
>	CUST0178	11100
>	CUST0133	11100
>	CUST0282	10600
>	CUST0269	10500
>	CUST0023	10450

20. Executive Dashboard Metrics

A single query returning five KPIs: total orders, unique customers, total revenue, average order value, and units sold.

```
SELECT
  COUNT(*) AS total_orders,
  COUNT(DISTINCT customer_id) AS customers,
  SUM(total_price) AS revenue,
  AVG(total_price) AS avg_order_value,
  SUM(quantity) AS units_sold
FROM sales_data;
```



	total_orders	customers	revenue	avg_order_value	units_sold
>	1000	292	1346600	1346.6	3008